

Hash File Generator and Verifier Using Flask

Introduction

The **File Integrity Verification System** is a web-based application developed using **Python Flask and PostgreSQL**. It is designed to generate a unique **SHA-256 hash** for uploaded files and store the hash along with the file information. The hash works as a digital fingerprint of the file.

The system allows users to upload and later verify their files by generating the hash again and comparing it with the original hash. If both hashes match, the file is considered unchanged; otherwise, the system indicates that the file has been modified. This project demonstrates the practical use of **Python, Flask, databases, file handling, and cybersecurity concepts**.

Files Format

file-integrity-verification/

|

|— app.py

|— requirements.txt

|— README.md

|— database_setup.sql

|— .env.example

|— .gitignore

|— SUBMISSION_CHECKLIST.md

|

|— templates/

| |— base.html

| |— index.html

| |— login.html

| |— register.html

| |— dashboard.html

```
| |— upload.html
| |— verify.html
|
|— static/
| |— css/
| | |— style.css
| |— js/
| | |— script.js
|
|— models/
| |— database.py
|
|— uploads/
| |— .gitkeep
|
|— screenshots/
| |— .gitkeep
```

Code: -

app.py

```
import os

import hashlib

from datetime import datetime

from functools import wraps


from flask import Flask, render_template, request, redirect, url_for, flash, send_from_directory

from flask_sqlalchemy import SQLAlchemy

from flask_login import LoginManager, UserMixin, login_user, logout_user, login_required,
current_user

from werkzeug.security import generate_password_hash, check_password_hash

from werkzeug.utils import secure_filename


app = Flask(__name__)

app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "change-this-secret-key")

app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
    "DATABASE_URL",
    "postgresql+psycopg2://postgres:password@localhost/file_integrity_db"
)

app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

app.config["UPLOAD_FOLDER"] = os.path.join(app.root_path, "uploads")

app.config["MAX_CONTENT_LENGTH"] = 16 * 1024 * 1024 # 16 MB


os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)


db = SQLAlchemy(app)
```

```
login_manager = LoginManager(app)
```

```
login_manager.login_view = "login"
```

```
class User(UserMixin, db.Model):
```

```
    __tablename__ = "users"
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    username = db.Column(db.String(80), unique=True, nullable=False)
```

```
    password_hash = db.Column(db.String(255), nullable=False)
```

```
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
```

```
    files = db.relationship("FileRecord", backref="owner", lazy=True, cascade="all, delete-orphan")
```

```
    activities = db.relationship("ActivityLog", backref="user", lazy=True, cascade="all, delete-orphan")
```

```
    def set_password(self, password):
```

```
        self.password_hash = generate_password_hash(password)
```

```
    def check_password(self, password):
```

```
        return check_password_hash(self.password_hash, password)
```

```
class FileRecord(db.Model):
```

```
    __tablename__ = "file_records"
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    original_filename = db.Column(db.String(255), nullable=False)
```

```
    stored_filename = db.Column(db.String(255), nullable=False)
```

```
    file_size = db.Column(db.BigInteger, nullable=False)
```

```
    sha256_hash = db.Column(db.String(64), nullable=False, index=True)
```

```
uploaded_at = db.Column(db.DateTime, default=datetime.utcnow)
owner_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
```

```
class ActivityLog(db.Model):
```

```
    __tablename__ = "activity_logs"
    id = db.Column(db.Integer, primary_key=True)
    action = db.Column(db.String(100), nullable=False)
    details = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id"), nullable=False)
```

```
@login_manager.user_loader
```

```
def load_user(user_id):
    return db.session.get(User, int(user_id))
```

```
def calculate_sha256(filepath):
    sha256 = hashlib.sha256()
    with open(filepath, "rb") as file:
        for block in iter(lambda: file.read(1024 * 1024), b''):
            sha256.update(block)
    return sha256.hexdigest()
```

```
def log_activity(action, details):
    db.session.add(ActivityLog(action=action, details=details, user_id=current_user.id))
    db.session.commit()
```

```
@app.route("/")
```

```
def index():
    if current_user.is_authenticated:
        return redirect(url_for("dashboard"))
    return render_template("index.html")

@app.route("/register", methods=["GET", "POST"])
def register():
    if current_user.is_authenticated:
        return redirect(url_for("dashboard"))

    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")

        if not username or not password:
            flash("Username and password are required.", "danger")
            return render_template("register.html")

        if len(password) < 6:
            flash("Password must contain at least 6 characters.", "danger")
            return render_template("register.html")

        if User.query.filter_by(username=username).first():
            flash("Username already exists.", "warning")
            return render_template("register.html")

        user = User(username=username)
```

```
user.set_password(password)
```

```
db.session.add(user)
```

```
db.session.commit()
```

```
flash("Registration successful. Please log in.", "success")
```

```
return redirect(url_for("login"))
```

```
return render_template("register.html")
```

```
@app.route("/login", methods=["GET", "POST"])
```

```
def login():
```

```
    if current_user.is_authenticated:
```

```
        return redirect(url_for("dashboard"))
```

```
    if request.method == "POST":
```

```
        username = request.form.get("username", "").strip()
```

```
        password = request.form.get("password", "")
```

```
        user = User.query.filter_by(username=username).first()
```

```
    if user and user.check_password(password):
```

```
        login_user(user)
```

```
        flash("Login successful.", "success")
```

```
        return redirect(url_for("dashboard"))
```

```
    flash("Invalid username or password.", "danger")
```

```
return render_template("login.html")
```

```
@app.route("/logout")
```

```
@login_required
```

```
def logout():
```

```
    logout_user()
```

```
    flash("You have been logged out.", "info")
```

```
    return redirect(url_for("index"))
```

```
@app.route("/dashboard")
```

```
@login_required
```

```
def dashboard():
```

```
    records =  
    FileRecord.query.filter_by(owner_id=current_user.id).order_by(FileRecord.uploaded_at.desc()).  
    all()
```

```
    activities =  
    ActivityLog.query.filter_by(user_id=current_user.id).order_by(ActivityLog.created_at.desc()).li  
    mit(10).all()
```

```
    return render_template("dashboard.html", records=records, activities=activities)
```

```
@app.route("/upload", methods=["GET", "POST"])
```

```
@login_required
```

```
def upload():
```

```
    if request.method == "POST":
```

```
        file = request.files.get("file")
```

```
        if not file or not file.filename:
```

```
            flash("Please select a file.", "danger")
```

```
            return render_template("upload.html")
```

```

original = secure_filename(file.filename)

if not original:

    flash("Invalid filename.", "danger")

    return render_template("upload.html")

# Use a unique server-side filename to avoid collisions and unsafe paths.

stored =
f"{datetime.utcnow().strftime('%Y%m%d%H%M%S%f')}_ {current_user.id}_ {original}"

path = os.path.join(app.config["UPLOAD_FOLDER"], stored)

file.save(path)

digest = calculate_sha256(path)

size = os.path.getsize(path)

record = FileRecord(

    original_filename=original,

    stored_filename=stored,

    file_size=size,

    sha256_hash=digest,

    owner_id=current_user.id

)

db.session.add(record)

db.session.commit()

log_activity("FILE_UPLOADED", f"{original} | SHA-256: {digest}")

flash("File uploaded and SHA-256 hash generated.", "success")

```

```

        return redirect(url_for("dashboard"))

    return render_template("upload.html")

@app.route("/verify/<int:file_id>", methods=["GET", "POST"])
@login_required
def verify(file_id):
    record = FileRecord.query.filter_by(id=file_id, owner_id=current_user.id).first_or_404()
    path = os.path.join(app.config["UPLOAD_FOLDER"], record.stored_filename)

    if not os.path.exists(path):
        flash("Stored file is missing. Verification cannot be completed.", "danger")
        return redirect(url_for("dashboard"))

    current_hash = calculate_sha256(path)
    is_valid = current_hash == record.sha256_hash

    if request.method == "POST":
        log_activity(
            "FILE_VERIFIED",
            f'{record.original_filename} | Expected: {record.sha256_hash} | Current: {current_hash}'
            | Result: {'VALID' if is_valid else 'MODIFIED'}"
        )
        flash(
            "Integrity verified: the file matches its stored SHA-256 hash." if is_valid
            else "Integrity warning: the file hash has changed.",
            "success" if is_valid else "danger"

```

```
)
```

```
return render_template(  
    "verify.html",  
    record=record,  
    current_hash=current_hash,  
    is_valid=is_valid  
)
```

```
@app.route("/download/<int:file_id>")  
@login_required  
def download(file_id):  
    record = FileRecord.query.filter_by(id=file_id, owner_id=current_user.id).first_or_404()  
    return send_from_directory(  
        app.config["UPLOAD_FOLDER"],  
        record.stored_filename,  
        as_attachment=True,  
        download_name=record.original_filename  
    )
```

```
@app.route("/delete/<int:file_id>", methods=["POST"])  
@login_required  
def delete_file(file_id):  
    record = FileRecord.query.filter_by(id=file_id, owner_id=current_user.id).first_or_404()  
    path = os.path.join(app.config["UPLOAD_FOLDER"], record.stored_filename)  
  
    if os.path.exists(path):
```

```
os.remove(path)
```

```
filename = record.original_filename
```

```
db.session.delete(record)
```

```
db.session.commit()
```

```
log_activity("FILE_DELETED", filename)
```

```
flash("File deleted successfully.", "info")
```

```
return redirect(url_for("dashboard"))
```

```
@app.errorhandler(413)
```

```
def too_large(_error):
```

```
    flash("File is too large. Maximum allowed size is 16 MB.", "danger")
```

```
    return redirect(url_for("upload"))
```

```
with app.app_context():
```

```
    db.create_all()
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Requirement.txt

```
Flask==3.1.2
```

```
Flask-Login==0.6.3
```

```
Flask-SQLAlchemy==3.1.1
```

```
psycpg2-binary==2.9.10
```

```
Werkzeug==3.1.3
```

Template/Base.html

```
<!doctype html>

<html lang="en">

<head>

    <meta charset="utf-8">

    <meta name="viewport" content="width=device-width, initial-scale=1">

    <title>{% block title %}File Integrity Verification{% endblock %}</title>

    <link          href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

    <link href="{{ url_for('static', filename='css/style.css') }}" rel="stylesheet">

</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">

    <div class="container">

        <a class="navbar-brand fw-bold" href="{{ url_for('index') }}">

            File Integrity System

        </a>

        <div>

            {% if current_user.is_authenticated %}

                <a class="btn btn-outline-light btn-sm me-2"

                    href="{{ url_for('dashboard') }}">

                        Dashboard

                </a>

                <a class="btn btn-outline-light btn-sm"

                    href="{{ url_for('logout') }}">

                        Logout

                </a>

            {% endif %}

        </div>

    </div>

</nav>
```

```

    {% else %}
        <a class="btn btn-outline-light btn-sm me-2"
            href="{{ url_for('login') }}">
            Login
        </a>
        <a class="btn btn-light btn-sm"
            href="{{ url_for('register') }}">
            Register
        </a>
    {% endif %}
</div>
</div>
</nav>
<main class="container py-4">
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% for category, message in messages %}
            <div class="alert alert-{{ category }} alert-dismissible fade show">
                {{ message }}
                <button type="button"
                    class="btn-close"
                    data-bs-dismiss="alert">
                </button>
            </div>
        {% endfor %}
    {% endwith %}
    {% block content %}
    {% endblock %}
</main>

```

```
<footer class="text-center text-muted py-4">
```

```
    M.Sc. IT Tiny Project — File Integrity Verification System
```

```
</footer>
```

```
<script
```

```
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

```
</body>
```

```
</html>
```

templates/index.html

```
{% extends "base.html" %}
```

```
{% block title %}Home{% endblock %}
```

```
{% block content %}
```

```
<div class="p-5 mb-4 bg-light rounded-3 shadow-sm">
```

```
    <h1 class="display-6 fw-bold">
```

```
        File Integrity Verification System
```

```
    </h1>
```

```
    <p class="lead">
```

```
        Generate SHA-256 hashes and verify whether uploaded files have been modified.
```

```
    </p>
```

```
    <a class="btn btn-dark btn-lg"
```

```
        href="{% url_for('register') %}">
```

```
        Get Started
```

```
    </a>
```

```
</div>
```

```
<div class="row g-4">
```

```
    <div class="col-md-4">
```

```
        <div class="card h-100 shadow-sm">
```

```
<div class="card-body">
  <h5>SHA-256</h5>
  <p>
    Generate a cryptographic hash for every uploaded file.
  </p>
</div>
</div>
</div>
<div class="col-md-4">
  <div class="card h-100 shadow-sm">
    <div class="card-body">
      <h5>Verification</h5>
      <p>
        Compare a new hash with the original stored hash.
      </p>
    </div>
  </div>
</div>
</div>
<div class="col-md-4">
  <div class="card h-100 shadow-sm">
    <div class="card-body">
      <h5>Audit Log</h5>
      <p>
        Track upload, verification, and deletion activity.
      </p>
    </div>
  </div>
</div>
```

</div>

</div>

{% endblock %}

templates/login.html

{% extends "base.html" %}

{% block title %}Login{% endblock %}

{% block content %}

<div class="row justify-content-center">

<div class="col-md-5">

<div class="card shadow-sm">

<div class="card-body p-4">

<h3 class="mb-4">

Login

</h3>

<form method="post">

<label class="form-label">

Username

</label>

<input
 class="form-control mb-3"
 name="username"
 required
>

<label class="form-label">
 Password
</label>

<input
 class="form-control mb-3"
 type="password"
 name="password"
 required
>

<button class="btn btn-dark w-100">
 Login
</button>

</form>

</div>

</div>

</div>

</div>

{% endblock %}

templates/register.html

{% extends "base.html" %}

{% block title %}Register{% endblock %}

{% block content %}

<div class="row justify-content-center">

<div class="col-md-5">

<div class="card shadow-sm">

<div class="card-body p-4">

<h3 class="mb-4">

Create Account

</h3>

```
<form method="post">
```

```
  <label class="form-label">
```

```
    Username
```

```
  </label>
```

```
  <input
```

```
    class="form-control mb-3"
```

```
    name="username"
```

```
    required
```

```
>
```

```
  <label class="form-label">
```

```
    Password
```

```
  </label>
```

```
  <input
```

```
    class="form-control mb-3"
```

```
    type="password"
```

```
    name="password"
```

```
    minlength="6"
```

```
    required
```

```
>
```

```
  <button class="btn btn-dark w-100">
```

```
    Register
```

```
  </button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

templates/dashboard.html

```
{% extends "base.html" %}
```

```
{% block title %}Dashboard{% endblock %}
```

```
{% block content %}
```

```
<div class="d-flex justify-content-between align-items-center mb-4">
```

```
<div>
```

```
<h2>
```

```
Dashboard
```

```
</h2>
```

<p class="text-muted mb-0">
Welcome, {{ current_user.username }}.
</p>

</div>

<a href="{{ url_for('upload') }}"
class="btn btn-dark">
Upload File

</div>

<div class="card shadow-sm mb-4">

<div class="card-body">

<h5>

Your Files

</h5>

<div class="table-responsive">

<table class="table align-middle">

<thead>

<tr>

<th>File</th>

<th>Size</th>

<th>SHA-256</th>

<th>Uploaded</th>

<th>Actions</th>

</tr>

</thead>

<tbody>

{% for r in records %}

<tr>

<td>

{{ r.original_filename }}

</td>

<td>

{{ "%.2f"|format(r.file_size / 1024) }} KB

</td>

<td>

<code class="small">

{{ r.sha256_hash }}

</code>

</td>

<td>

{{ r.uploaded_at.strftime('%Y-%m-%d %H:%M') }}

</td>

<td class="text-nowrap">

<a

class="btn btn-sm btn-outline-primary"

href="{{ url_for('verify', file_id=r.id) }}">

Verify

<a

class="btn btn-sm btn-outline-secondary"

href="{{ url_for('download', file_id=r.id) }}">

Download

<form

class="d-inline"

```
method="post"
action="{{ url_for('delete_file', file_id=r.id) }}"
onsubmit="return confirm('Delete this file?');">
```

```
<button class="btn btn-sm btn-outline-danger">
    Delete
</button>
```

```
</form>
```

```
</td>
```

```
</tr>
```

```
{% else %}
```

```
<tr>
```

```
<td colspan="5"
    class="text-center text-muted">
```

No files uploaded yet.

```
</td>
```

```
</tr>
```

```
{% endfor %}
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="card shadow-sm">
```

```
<div class="card-body">
```

```
<h5>
```

Recent Activity

```
</h5>
```

```
<ul class="list-group list-group-flush">
```

```
{% for a in activities %}
```

```
<li class="list-group-item px-0">
```


{{ a.action }}

—

{{ a.details }}

{{ { a.created_at.strftime('%Y-%m-%d %H:%M') }}

{% else %}

<li class="list-group-item px-0 text-muted">

No activity yet.

{% endfor %}

</div>

```
</div>
```

```
{% endblock %}
```

templates/upload.html

```
{% extends "base.html" %}
```

```
{% block title %}Upload{% endblock %}
```

```
{% block content %}
```

```
<div class="row justify-content-center">
```

```
<div class="col-md-7">
```

```
<div class="card shadow-sm">
```

```
<div class="card-body p-4">
```

```
<h3>
```

Upload File

```
</h3>
```

```
<p class="text-muted">
```

Maximum file size: 16 MB.

```
</p>
```

```
<form method="post"
      enctype="multipart/form-data">
```

```
<input
      class="form-control mb-3"
      type="file"
      name="file"
      required
    >
```

```
<button class="btn btn-dark">
      Upload & Generate SHA-256
</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

templates/verify.html

```
{% extends "base.html" %}
```

```
{% block title %}Verify File{% endblock %}
```

```
{% block content %}
```

```
<div class="card shadow-sm">
```

```
<div class="card-body p-4">
```

```
<h3>
```

```
    Integrity Verification
```

```
</h3>
```

```
<p>
```

```
    <strong>File:</strong>
```

```
    {{ record.original_filename }}
```

```
</p>
```

```
<p>
```

```
    <strong>
```

```
        Stored SHA-256:
```

```
    </strong>
```

```
<br>
```

```
<code class="hash">
    {{ record.sha256_hash }}
</code>
```

</p>

<p>

```
<strong>
    Current SHA-256:
</strong>
```



```
<code class="hash">
    {{ current_hash }}
</code>
```

</p>

{% if is_valid %}

<div class="alert alert-success">

VALID:

The current file matches the stored SHA-256 hash.

</div>

{% else %}

<div class="alert alert-danger">

MODIFIED:

The current file does not match the stored SHA-256 hash.

</div>

{% endif %}

<form method="post">

<button class="btn btn-dark">

Record Verification in Audit Log

</button>

<a

class="btn btn-outline-secondary"

href="{{ url_for('dashboard') }}">

Back

</form>

</div>

</div>

{% endblock %}

static/css/style.css

```
body {  
    background: #f7f7f8;  
}
```

```
.card {  
    border: 0;  
}
```

```
.hash {  
    word-break: break-all;  
}
```

```
code {  
    word-break: break-all;  
}
```

```
footer {  
    font-size: 0.9rem;  
}
```

Output

```
pip install -r requirements.txt
```

```
python app.py
```

```
http://127.0.0.1:5000
```